

EJERCICIO GUIADO + TRABAJO EN CASA

Clasificador de modelos de servicios Cloud con Inteligencia Artificial

Java | Python | GUI | CLI | Regex | NLP | Arquitectura Cloud | Prompt Engineering

Nombre	
Matrícula	
Grupo y hora	
Fecha	

EJERCICIO GUIADO

Objetivo:

Desarrollar, con apoyo de herramientas de Inteligencia Artificial generativa, una aplicación capaz de analizar palabras, frases o descripciones relacionadas con servicios de Cloud Computing y clasificarlas automáticamente como IaaS, PaaS, SaaS o FaaS. El ejercicio permitirá analizar código generado por IA y evolucionar desde un clasificador basado en reglas hacia una solución básica de Procesamiento de Lenguaje Natural (NLP).

- IaaS — Infrastructure as a Service
- PaaS — Platform as a Service
- SaaS — Software as a Service
- FaaS — Function as a Service

Parte 1. Desarrollo inicial en Java

1. Crear el proyecto

Utiliza el IDE moderno o editor de código de tu preferencia, por ejemplo Visual Studio Code, IntelliJ IDEA, Eclipse, Cursor u otro entorno compatible con Java.

Crea un proyecto llamado: `cloud_models_classifier`

La primera versión deberá ejecutarse como una aplicación de escritorio con interfaz gráfica (GUI).

2. Crear el primer prototipo con IA generativa

Utiliza el siguiente prompt como punto de partida:

Crea una aplicación de escritorio en Java con interfaz gráfica (GUI). La aplicación debe solicitar el nombre y apellido del usuario y proporcionar un campo de texto donde pueda escribir palabras, frases o una descripción relacionada con un servicio de Cloud Computing.

La aplicación deberá analizar el texto introducido y determinar si corresponde principalmente a uno de los siguientes modelos de servicios Cloud: IaaS, PaaS, SaaS o FaaS.

Para esta primera versión utiliza reglas sencillas, palabras clave y expresiones regulares (Regex). Muestra el resultado de la clasificación en una sección claramente visible de la interfaz.

Organiza el código utilizando funciones o métodos independientes para cada categoría y agrega comentarios que expliquen las principales partes de la solución.

Importante: no copies el código generado sin analizarlo. Debes comprender la propuesta de la IA, identificar sus componentes y realizar las modificaciones necesarias para poder explicar cómo funciona.

Parte 2. Ejecutar y analizar el prototipo

3. Ejecutar la aplicación

Compila y ejecuta la aplicación en la computadora de escritorio del laboratorio. Verifica que:

- La GUI se muestre correctamente.
- Sea posible introducir nombre y apellido.
- El usuario pueda introducir una palabra, frase o descripción.
- El programa analice el contenido.
- Se muestre claramente la clasificación obtenida.
- La aplicación distinga entre IaaS, PaaS, SaaS y FaaS.

Realiza screenshots que documenten el funcionamiento.

Parte 3. Mejorar la arquitectura de la aplicación

4. Refactorizar el código

Analiza el código generado por la IA y mejora su estructura. Como mínimo incorpora:

- Validación de entradas.
- Separación entre GUI y lógica de clasificación.
- Métodos independientes para identificar IaaS, PaaS, SaaS y FaaS.
- Manejo básico de errores y excepciones.
- Comentarios donde ayuden a comprender decisiones relevantes.

La GUI no deberá contener directamente toda la lógica del clasificador.

Parte 4. Evaluar el clasificador

5. Crear casos de prueba

Prueba la aplicación con al menos cinco descripciones diferentes. Evita utilizar únicamente las palabras IaaS, PaaS, SaaS o FaaS.

Ejemplo A: Necesito máquinas virtuales, almacenamiento y redes configurables para instalar mi propio sistema operativo. **Resultado esperado:** IaaS

Ejemplo B: Quiero desplegar mi aplicación web sin administrar directamente servidores ni sistemas operativos. **Resultado esperado:** PaaS

Ejemplo C: Los empleados utilizan una aplicación de correo electrónico directamente desde el navegador y pagan una suscripción mensual. **Resultado esperado:** SaaS

Ejemplo D: Necesito ejecutar una función automáticamente cada vez que un usuario suba una imagen al almacenamiento Cloud. **Resultado esperado:** FaaS

Registra cada prueba con el formato:

Entrada → Clasificación esperada → Clasificación obtenida → ¿Fue correcta?

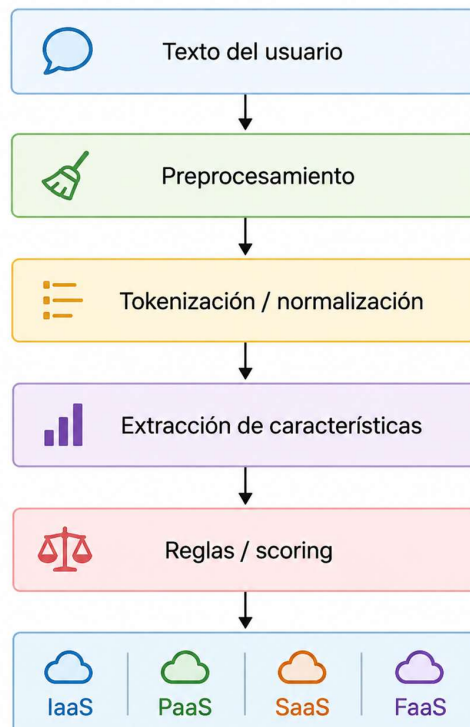
Si encuentras errores, modifica las reglas y repite las pruebas.

Parte 5. Evolucionar de Regex hacia NLP

6. Incorporar procesamiento básico de lenguaje natural

Mejora el clasificador utilizando técnicas básicas de Natural Language Processing (NLP):

- Conversión a minúsculas.
- Limpieza de texto.
- Tokenización.
- Eliminación de stopwords.
- Normalización.
- Stemming o lematización, cuando sea apropiado.
- Identificación de palabras o conceptos relevantes.
- Asignación de puntuaciones (scores) a cada categoría.



Flujo simplificado del clasificador basado en NLP y reglas.

En esta etapa no utilices directamente un LLM para responder la categoría. El objetivo es comprender cómo el procesamiento del lenguaje mejora un clasificador basado en reglas.

Parte 6. Interfaz de línea de comandos en Java

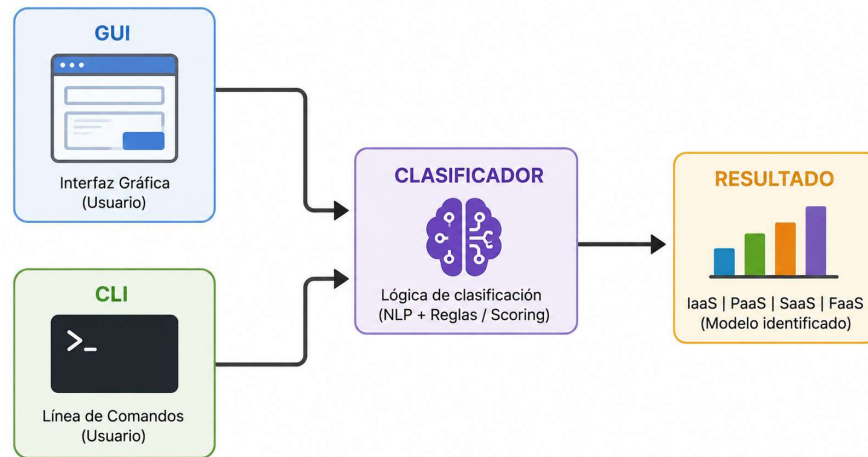
7. Crear una versión CLI

Además de la GUI, permite utilizar el clasificador desde línea de comandos. La GUI y la CLI deberán reutilizar la misma lógica.

```
java CloudClassifier "máquinas virtuales almacenamiento redes"
```

Modelo identificado: IaaS

La arquitectura esperada deberá aproximarse a:



TRABAJO EN CASA

Las siguientes actividades se realizarán de manera individual fuera del laboratorio. Todas las evidencias deberán integrarse al sitio web personal provisto por el profesor, creando una página independiente para cada tarea.

Tarea 1. Implementación del clasificador Cloud en Python

Objetivo

Reimplementar y mejorar en Python el clasificador desarrollado en Java, aplicando separación de responsabilidades, NLP básico y reutilización de componentes.

Actividad

Desarrolla una versión en Python con GUI. Puedes investigar Tkinter, CustomTkinter, PySide o PyQt.

- Captura de nombre y apellido.
- Entrada de texto.
- Clasificación IaaS/PaaS/SaaS/FaaS.
- Validación de datos.
- Manejo de errores.
- Separación entre GUI y lógica.
- Funciones o módulos independientes.
- Procesamiento básico de lenguaje natural.
- Presentación clara del resultado.

Interfaz CLI

Implementa también una CLI utilizando argparse. GUI y CLI deberán compartir la misma lógica de clasificación.

```
python classifier.py --text "ejecutar una función cuando se suba una imagen"
```

```
Modelo identificado: FaaS
```

Pruebas y evidencias

Realiza al menos 5 pruebas y documenta:

```
Entrada → Resultado esperado → Resultado obtenido → Correcto/Incorrecto
```

- Descripción del ejercicio.
- Arquitectura.
- Código fuente.
- Explicación del NLP.
- Screenshots GUI y CLI.
- Tabla de 5 casos.

- Errores encontrados.
- Mejoras realizadas.
- Conclusiones.

Tarea 2. Mapa conceptual comparativo de modelos Cloud

Objetivo

Comprender y representar visualmente las diferencias entre IaaS, PaaS, SaaS y FaaS y relacionarlas con servicios reales.

Actividad

Utiliza Miro, Lucidchart, Canva u otra herramienta de diagramación para crear un mapa conceptual comparativo.

Para cada modelo incluye:

- Definición.
- Características principales.
- Qué administra el proveedor.
- Qué administra el cliente.
- Ventajas y desventajas.
- Casos de uso.
- Nivel de control.
- Nivel de abstracción.
- Escalabilidad.
- Ejemplos reales.

Investiga servicios actuales de AWS, Microsoft Azure, Google Cloud, Oracle Cloud Infrastructure (OCI) u otros proveedores relevantes. No te limites a colocar logotipos: identifica servicios específicos y explica por qué pertenecen a cada modelo.

El mapa deberá representar la relación:

Mayor control del usuario → IaaS → PaaS → SaaS → Mayor abstracción

Analiza dónde ubicarías FaaS/Serverless y justifica tu decisión.

Evidencia

- Mapa conceptual completo.
- Imagen en alta resolución o versión embebida.
- Explicación breve.
- Fuentes consultadas.
- Enlace al mapa original, cuando corresponda.

Tarea 3. Miniensayo: Cloud-Native vs. Cloud-Enabled

Tema

“¿Cloud-Native o Cloud-Enabled? Análisis de una aplicación real.”

Actividad

Selecciona una aplicación, plataforma o sistema real y determina, con argumentos técnicos, si puede considerarse Cloud-Native, Cloud-Enabled o una combinación de ambos.

Considera, cuando sean relevantes:

- Arquitectura monolítica vs. microservicios.
- Contenedores y orquestación.
- APIs.
- Serverless/FaaS.
- Bases de datos administradas.
- Escalabilidad horizontal y elasticidad.
- Alta disponibilidad.
- Observabilidad.
- DevOps y CI/CD.
- Infraestructura como código.
- Tolerancia a fallos.

Extensión aproximada: 600 palabras.

Introducción → Aplicación seleccionada → Análisis tecnológico → Argumentación Cloud-Native/Cloud-Enabled → Conclusiones

Utiliza referencias actuales y confiables: Biblioteca digital UDEM, IEEE, ACM Digital Library, documentación oficial de proveedores Cloud, libros especializados y publicaciones técnicas reconocidas.

Evidencia

- Miniensayo completo.
- Aplicación seleccionada.
- Imágenes o diagramas relevantes.
- Referencias bibliográficas.
- Conclusión personal.

Tarea 4. Arquitectura Cloud con Prompt Engineering y agentes de IA

Objetivo

Utilizar un IDE moderno con capacidades de IA generativa o agentes de programación para diseñar, analizar y prototipar una arquitectura Cloud, evaluando críticamente la solución propuesta.

Puedes utilizar Visual Studio Code con GitHub Copilot, Cursor, Windsurf, JetBrains con herramientas de IA u otro entorno equivalente.

Escenario

Una institución financiera desea desarrollar una nueva aplicación de banca móvil. La solución deberá considerar:

- Aplicación móvil.
- Backend y APIs.
- Autenticación.
- Base de datos y almacenamiento.
- Notificaciones.
- Funciones serverless.
- Alta disponibilidad y escalabilidad.
- Seguridad y monitoreo.
- Integración con infraestructura existente.
- Arquitectura de nube híbrida.

Prompt inicial sugerido

Actúa como arquitecto de soluciones Cloud. Diseña una arquitectura para una aplicación de banca móvil utilizando una estrategia de nube híbrida.

La solución deberá incluir aplicación móvil, API Gateway, servicios backend, autenticación, bases de datos, almacenamiento, funciones serverless/FaaS, monitoreo y mecanismos de seguridad.

Identifica qué componentes deberían permanecer on-premises y cuáles deberían ejecutarse en la nube, justificando cada decisión.

Propón una arquitectura escalable, tolerante a fallos y altamente disponible.

Genera un diagrama de arquitectura y desarrolla un prototipo básico del backend. Explica cada componente y las decisiones arquitectónicas realizadas.

Prompt Engineering

No aceptes automáticamente la primera respuesta. Utiliza al menos cinco prompts adicionales para mejorar aspectos específicos.

- Identificar vulnerabilidades.
- Mejorar arquitectura.

- Reducir puntos únicos de falla.
- Mejorar escalabilidad.
- Justificar servicios.
- Incorporar observabilidad.
- Analizar costos.
- Proponer recuperación ante desastres.
- Detectar problemas del código.
- Generar pruebas.
- Revisar la arquitectura desde seguridad.

Documenta cómo cambia la solución después de cada iteración.

Diagrama arquitectónico

El diagrama deberá mostrar claramente:

Usuario → Aplicación móvil → Servicios Cloud → Backend → Datos → Sistemas on-premises

- Frontera Cloud.
- Frontera on-premises.
- Comunicación entre entornos.
- APIs.
- Componentes serverless.
- Bases de datos.
- Seguridad.
- Monitoreo.

Código

Incluye el prototipo de backend generado y mejorado. Debes poder explicar qué hace, cómo funciona, por qué fue diseñado así y qué modificarías para producción.

Reflexión individual (aprox. 500 palabras)

- Prompt Engineering: ¿cómo cambió la solución conforme mejoraste tus instrucciones?
- Arquitectura: ¿la primera propuesta era adecuada?
- Errores: ¿detectaste omisiones o decisiones cuestionables?
- Seguridad: ¿la IA consideró adecuadamente el contexto financiero?
- Código: ¿comprendes completamente el código generado?
- Validación: ¿cómo verificaste las recomendaciones?
- Rol del ITC: ¿qué conocimientos necesita un ingeniero para validar una arquitectura generada por IA?
- IA vs. ingeniero: ¿qué decisiones pueden delegarse y cuáles deben permanecer bajo responsabilidad humana?
- Conclusión: ¿una persona sin bases sólidas de arquitectura, programación, datos, redes y seguridad podría evaluar correctamente esta solución sólo con IA?

Evidencias de la Tarea 4

- Descripción del problema.
- Herramienta de IA utilizada.
- Prompt inicial.
- Al menos cinco prompts de mejora.
- Primera arquitectura.
- Arquitectura final.
- Diagrama.
- Código generado y mejorado.
- Screenshots.
- Errores detectados.
- Mejoras realizadas.
- Reflexión de aproximadamente 500 palabras.
- Conclusiones.

Entrega del trabajo en casa

Genera tu sitio web personal en ubiquitous.udem.edu, y asegúrate de que tu primer tarea contenga una sección claramente identificable para cada actividad:

- Tarea 1 → Clasificador Cloud en Python.
- Tarea 2 → Mapa Conceptual IaaS / PaaS / SaaS / FaaS.
- Tarea 3 → Miniensayo Cloud-Native vs. Cloud-Enabled.
- Tarea 4 → Arquitectura Cloud + Prompt Engineering.

Consideración sobre el uso de Inteligencia Artificial

Se permite y fomenta el uso de herramientas de IA, pero su utilización deberá documentarse. El estudiante continúa siendo responsable de:

comprender → revisar → probar → validar → corregir → justificar

El objetivo no es evaluar quién puede generar más contenido mediante prompts, sino quién demuestra mayor capacidad para utilizar la IA como herramienta de ingeniería y evaluar críticamente sus resultados.